DATABASE SCHEMA

& ERD DOCUMENTATION

E-Commerce Platform

Firebase Firestore Architecture

Generated: April 29, 2026

Table of Contents

1. Project Overview

This document describes the database schema and entity relationships for the E-Commerce Platform built with Firebase Firestore. The system is designed to support a scalable multi-category product marketplace with user authentication, product management, and comprehensive order tracking.

Technology Stack

- **Database:** Firebase Firestore (NoSQL document database)
- **Storage:** Cloudinary for image hosting and CDN delivery
- **Authentication:** Firebase Authentication with Google OAuth2
- **Backend:** Node.js/Express server
- **Password Security:** bcryptjs hashing (10 salt rounds)
- **Session Management:** Express sessions with custom middleware

Project Scope

The platform manages users, products, and orders across multiple product categories including phones, laptops, accessories, headphones, watches, and kitchen equipment. The system handles user registration, authentication, product browsing, shopping cart management, order processing, and order tracking with real-time status updates.

2. Database Architecture

The system uses a NoSQL document-based approach leveraging Firebase Firestore's scalable infrastructure. Unlike traditional relational databases, Firestore organizes data into collections and documents, enabling flexible schema evolution and real-time synchronization.

Collections Overview

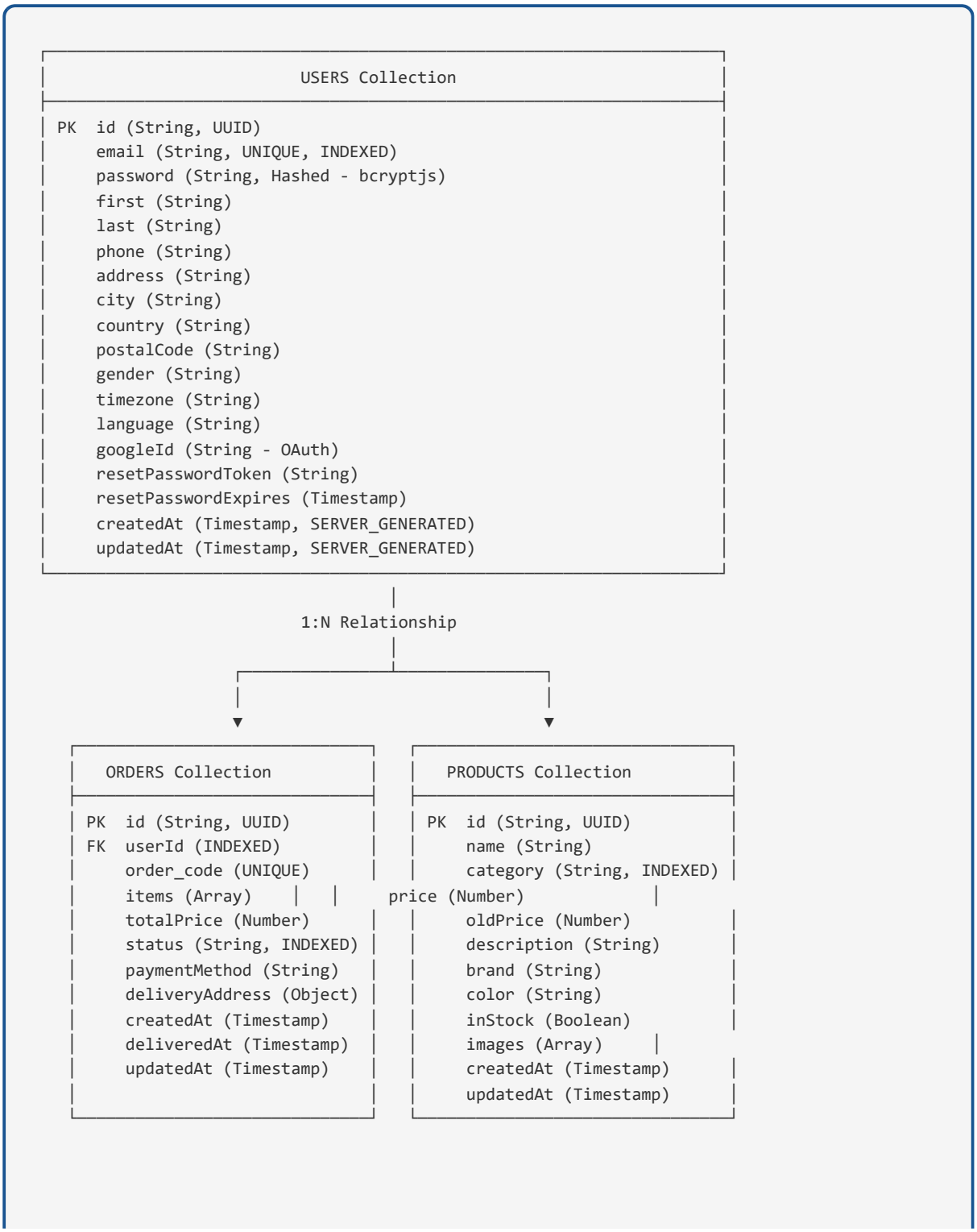
- **Users:** Manages user accounts, authentication credentials, and profile information
- **Products:** Maintains product inventory, pricing, descriptions, and availability status
- **Orders:** Tracks all transactions with order items, status workflow, and delivery details

Architecture Principles

- Document-oriented storage for flexible field definitions
- Denormalization where appropriate to reduce query complexity
- Server-side timestamp generation for consistency
- Indexed fields for efficient querying and filtering
- Relationship management through document references (foreign keys)

3. Entity Relationship Diagram

The following diagram illustrates the relationships between the three main collections:



Cardinality: One User has Many Orders. Each Order references one User through userId.

4. Collections Details

4.1 Users Collection

Stores user account information, authentication credentials, and profile details. Each document represents a unique user account in the system.

Field Name	Data Type	Constraints	Description
id	String	PK, UUID	Unique user identifier
email	String	UNIQUE, INDEXED, Required	User email address (stored lowercase)
password	String	Hashed, Required	Bcryptjs hashed password (never plain text)
first	String	Optional	First name
last	String	Optional	Last name
phone	String	Optional	Contact phone number
address	String	Optional	Street address

Field Name	Data Type	Constraints	Description
city	String	Optional	City name
country	String	Optional	Country name
postalCode	String	Optional	Postal/ZIP code
gender	String	Optional	Gender (male, female, other)
timezone	String	Optional	User timezone (e.g., UTC+2)
language	String	Optional	Preferred language (en, ar, etc.)
googleId	String	Optional	Google OAuth ID for social login
resetPasswordToken	String	Optional	Token for password reset functionality
resetPasswordExpires	Timestamp	Optional	Token expiration time
createdAt	Timestamp	Required, SERVER_GENERATED	Account creation timestamp
updatedAt	Timestamp	Required, SERVER_GENERATED	Last account update timestamp

4.2 Products Collection

Manages the product catalog with inventory status, pricing information, descriptions, and image references. Products are organized by category and support discount pricing through oldPrice field.

Field Name	Data Type	Constraints	Description
id	String	PK, UUID	Unique product identifier
name	String	Required	Product name/title
category	String	INDEXED, Required	Product category (phones, laptops, accessories, headphones, watches, kitchen)
price	Number	Required	Current selling price
oldPrice	Number	Optional	Original price (for showing discounts)
description	String	Optional	Detailed product description
brand	String	Optional	Product brand/manufacturer
color	String	Optional	Product color
inStock	Boolean	Default: true	Inventory availability status

Field Name	Data Type	Constraints	Description
images	Array<String>	Optional	Array of Cloudinary image URLs
createdAt	Timestamp	SERVER_GENERATED	Product creation timestamp
updatedAt	Timestamp	SERVER_GENERATED	Last product update timestamp

4.3 Orders Collection

Tracks all customer orders with items, pricing, payment method, delivery information, and status workflow. Each order references a user and maintains a snapshot of product prices at purchase time.

Field Name	Data Type	Constraints	Description
id	String	PK, UUID	Unique order identifier
order_code	String	UNIQUE, Human-readable	Order reference code (6-char alphanumeric)
userId	String	FK, INDEXED, Required	Reference to Users collection
items	Array<Object>	Required	Ordered items with product details, quantity, and price at purchase
totalPrice	Number	Required	Total order amount
status	String	INDEXED, Enum	Order status: pending, processing, shipped, delivered, cancelled
paymentMethod	String	Optional	Payment method (credit card, bank transfer, etc.)

Field Name	Data Type	Constraints	Description
deliveryAddress	Object	Optional	Delivery address with street, city, postal code
createdAt	Timestamp	SERVER_GENERATED	Order creation timestamp
deliveredAt	Timestamp	Optional	Delivery completion timestamp
updatedAt	Timestamp	SERVER_GENERATED	Last status update timestamp

5. Data Types Reference

The following table describes all data types used in the schema:

Data Type	Description	Example
String	Text data, variable length	"John Doe", "john@example.com"
Number	Integer or floating-point values	299.99, 5, -100
Boolean	True or false values	true, false
Timestamp	Date and time values (Firebase FieldValue.serverTimestamp)	2026-04-29T14:30:00Z
Array	Ordered collection of values (homogeneous or mixed)	["image1.jpg", "image2.jpg"]
Object	Key-value pairs (nested documents)	{ street: "123 Main St", city: "Cairo" }

Status Enumerations

Order Status Values:

Status	Description	Allowed Transitions
pending	Order awaiting payment and processing	→ processing, cancelled
processing	Order being prepared for shipment	→ shipped, cancelled
shipped	Order in transit to customer	→ delivered
delivered	Order successfully delivered	→ (terminal state)
cancelled	Order cancelled by customer or system	(terminal state)

6. Relationships & Constraints

Primary Keys (PK)

- **Users.id** – Unique identifier for each user (UUID format)
- **Products.id** – Unique identifier for each product (UUID format)
- **Orders.id** – Unique identifier for each order (UUID format)

Foreign Keys (FK)

- **Orders.userId → Users.id** – Establishes one-to-many relationship (One User can have many Orders)

Unique Constraints

- **Users.email** – Must be globally unique; prevents duplicate user accounts
- **Orders.order_code** – Must be unique; used for customer reference

Indexed Fields

- **Users.email** – Accelerates login queries
- **Products.category** – Optimizes category browsing
- **Orders.userId** – Enables efficient user order retrieval
- **Orders.status** – Facilitates status-based queries and filtering
- **Orders.createdAt** – Supports chronological ordering

Data Integrity Rules

- User email addresses are automatically stored in lowercase for consistency
- All system timestamps are server-generated using Firebase FieldValue.serverTimestamp() to prevent client-side clock skew

- Passwords are hashed using bcryptjs with 10 salt rounds; plain passwords are never stored
- Product prices are stored as numbers (recommend storing in cents to avoid floating-point errors)
- Order status follows a defined workflow: pending → processing → shipped → delivered, with cancellation as an alternative path
- Product images reference external Cloudinary URLs; no images are stored in Firestore

7. Implementation Notes

Authentication & Security

Users authenticate via email/password combination or Google OAuth2. Passwords are hashed using bcryptjs with a salt round of 10, providing strong resistance to brute-force attacks. Password reset functionality uses time-limited tokens stored in the resetPasswordToken and resetPasswordExpires fields. These tokens are cleared upon successful password reset.

Image Storage & Delivery

Product images are hosted on Cloudinary, a dedicated image hosting and CDN service. The images array in Products contains URLs to these resources. This approach provides:

- Unlimited scalability without Firestore storage constraints
- Automatic optimization and resizing
- Global CDN for fast delivery
- Separation of concerns between data and media

Order Processing Workflow

Orders follow a state machine pattern:

1. **Pending:** Order created, awaiting payment confirmation
2. **Processing:** Payment confirmed, order being prepared
3. **Shipped:** Order handed to shipping carrier
4. **Delivered:** Delivery confirmed at destination

The items array maintains a snapshot of product data at purchase time (name, price, quantity), protecting against future price changes affecting historical orders.

Query Patterns

Common queries are optimized through strategic indexing:

- Find user by email: Uses Users.email index
- List products by category: Uses Products.category index
- Get user's orders: Uses Orders.userId index with createdAt sort

- Filter orders by status: Uses Orders.status index

Scalability Considerations

Firebase Firestore automatically scales to handle millions of documents and provides:

- Real-time synchronization for collaborative features
- Automatic sharding and distribution
- ACID transactions for multi-document updates
- Read and write rate scaling
- Backup and disaster recovery

Data Migration & Versioning

The schema supports evolution through:

- Optional fields that don't break existing documents

- Gradual addition of new collections
- Backward-compatible field additions
- Migration scripts for large-scale updates

Performance Optimization

- Lean queries return plain objects instead of instantiated classes for read-heavy operations
- Server-side sorting for consistent ordering across clients
- Selective field projection to minimize data transfer
- Batch operations for bulk updates

Backup & Recovery

Firebase Firestore provides automated daily backups. For critical operations, consider:

- Regular data exports to Cloud Storage
- Point-in-time recovery capabilities

- Audit logging for compliance

Document Information

Document Title: Database Schema & ERD Documentation

Project: E-Commerce Platform

Database System: Firebase Firestore

Generated Date: April 29, 2026

Version: 1.0

Language: English

Changelog

Version 1.0 (April 29, 2026): Initial comprehensive schema documentation with ERD, field definitions, constraints, and implementation guidelines.

This document is a comprehensive guide to the database architecture of the E-Commerce Platform. For inquiries or updates, please consult with the development team.